

OAT 1

Web API Fundacional da MecâniQA

Autores: Alex Gabriel Alves dos Santos
Gustavo Azevedo Diniz
Gustavo Muniz dos Anjos
João Paulo Rosa Batista

Agenda

Objetivo

Explicar como o MVP gerencia peças e serviços aplicando apenas os conceitos exigidos na OAT 1.

1. Contexto e MVP

Problema e oito user stories

2. Conceitos de OO

Classes, encapsulamento e enum

3. Controllers REST

Rotas, verbos e status HTTP

4. Padrão Singleton

Estado em memória sem injeção

5. Síntese da entrega

Código, collection e UML

O MVP entrega o CRUD fundacional da oficina

Problema

A MecâniQA precisa migrar o gerenciamento de peças e serviços para uma Web API.

8 user stories

CRUD completo de Peça e Serviço



Sem banco de dados, Spring Data ou @Autowired: a persistência desta fase fica em listas na memória.

Encapsulamento organiza as entidades

```
class Peca {  
    - codigo : Long  
    - precoVenda : BigDecimal  
    - categoria : CategoriaPeca  
  
    + getCodigo() : Long  
    + setPrecoVenda(...) : void  
}  
  
enum CategoriaPeca {  
    MOTOR, SUSPENSAO, FREIOS,  
    ELETRICA, ACESSORIOS  
}
```

Classes e atributos

Peca e Servico reúnem estado e comportamento; seus atributos são private.

Construtores

Criam os objetos. Código e datas são preenchidos pela aplicação no cadastro.

Getters e setters

Métodos public permitem consultar ou alterar os campos encapsulados.

Enum

CategoriaPeca limita a categoria aos cinco valores definidos no catálogo.

static

O membro pertence à classe; no Singleton, mantém a instância única e o getInstance().

Controllers ligam verbos HTTP às respostas semânticas

@RestController + @RequestMapping

/api/pecas e /api/servicos

POST	cadastrar	201 Created
GET	listar / buscar	200 OK; 404 na busca
PUT	atualizar	200 OK ou 404
DELETE	excluir	204 No Content ou 404

Por que ResponseEntity?

- Retorna o corpo e o status HTTP adequado.
- 201 confirma criação; 200 confirma consulta ou atualização.
- 204 confirma exclusão sem corpo de resposta.
- 404 informa que o código solicitado não existe.

Singleton mantém uma instância única por repository



- Construtor private impede criar outro repository com new.
- `getInstance()` static devolve sempre o mesmo objeto.
- Atributo static guarda a única instância da classe.
- As listas sobrevivem na memória enquanto a aplicação roda.

A entrega cobre as oito histórias sem ampliar o escopo

8/8

user stories implementadas

- ✓ Peca, Servico e CategoriaPeca
- ✓ Requests com os corpos das requisições
- ✓ Repositories Singleton em memória
- ✓ Controllers REST com ResponseEntity
- ✓ Collection Postman executável
- ✓ UML conferido com código e static

No diagrama, nomes, tipos, visibilidades e membros static correspondem ao código.

- OAT 1 - Introdução a Diagrama de Classes.pdf
- Baremas - Desenvolvimento Web Orientado a Objetos - OAT 1
- CADERNO DE GUIAS - OAT 1 - TUTORIAIS.pdf
- Modelo de Apresentação.pdf
- Repositório mecaniQA-api-rio-de-janeiro: código, collection e UML

Material preparado para a avaliação da OAT 1.